Fondamenti LLM & Agentic AI

Corso operativo in italiano · Manuale, laboratori e copioni video

Materiale didattico personalizzato

Ottobre 2026

Indice

1 Come usare questo manuale **3**

2 Modulo 1 — Fondamenti degli LLM **3**
2.1 Obiettivi . . . 3
2.2 1.1 Che cosa apprende un LLM . . . 3
2.3 1.2 Dal testo alla previsione . . . 3
2.4 1.3 Decodifica e parametri . . . 4
2.5 1.4 La risposta del nostro Qwen3 . . . 4
2.6 Esperimento . . . 4
2.7 Autoverifica . . . 4
2.8 Video suggerito . . . 4

3 Modulo 2 — Local LLM & Model Serving **4**
3.1 2.1 Esecuzione locale: componenti . . . 4
3.2 2.2 VRAM: non bastano i parametri . . . 5
3.3 2.3 Prefill, decode, KV cache, batching . . . 5
3.4 2.4 Laboratorio consigliato . . . 5
3.5 2.5 Produzione . . . 5
3.6 Autoverifica . . . 5
3.7 Video suggeriti . . . 6

4 Modulo 3 — Retrieval-Augmented Generation e Knowledge Base **6**
4.1 3.1 Che problema risolve . . . 6
4.2 3.2 Pipeline di ingestione . . . 6
4.3 3.3 Query time . . . 6
4.4 3.4 Evaluation: una condizione per parlare di produzione . . . 7
4.5 3.5 Laboratorio . . . 7
4.6 Autoverifica . . . 7
4.7 Video suggerito . . . 7

5 Modulo 4 — Fine-Tuning, LoRA e QLoRA **7**
5.1 4.1 Quando scegliere il fine-tuning . . . 7
5.2 4.2 Tecniche . . . 7
5.3 4.3 Dataset e qualità . . . 8
5.4 4.4 Laboratorio . . . 8
5.5 Autoverifica . . . 8
5.6 Video suggerito . . . 8

6 Modulo 5 — Agentic AI, tool calling, ReAct, LangGraph e MCP **8**
6.1 5.1 Definizioni da non confondere . . . 8
6.2 5.2 Stato e grafo . . . 9
6.3 5.3 Tool contracts e structured outputs . . . 9
6.4 5.4 MCP (Model Context Protocol) . . . 9
6.5 5.5 Agentic RAG . . . 9
6.6 5.6 Laboratorio . . . 9
6.7 Autoverifica . . . 9

6.8 Videotutorial 10

7 Modulo 6 — Platform, Security e Human-in-the-Loop 10
7.1 6.1 Piattaforma: dove si trova ogni componente 10
7.2 6.2 Fiducia tra componenti 10
7.3 6.3 Human-in-the-Loop (HITL) 10
7.4 6.4 Esercizio pratico 11
7.5 Autoverifica 11
7.6 Videotutorial 11

8 Modulo 7 — Prompt Caching, Prefix Caching, KV Cache e costi 11
8.1 7.1 Tre cache diverse 11
8.2 7.2 Automatic Prefix Caching in vLLM 11
8.3 7.3 Bedrock e provider gestiti 12
8.4 7.4 Errori concettuali frequenti 12
8.5 7.5 Laboratorio 12
8.6 Autoverifica 12
8.7 Videotutorial 12

9 Modulo 8 — Troubleshooting, osservabilità e quality assurance 12
9.1 8.1 Diagnostica per livelli 12
9.2 8.2 Problemi incontrati nell'esercitazione 13
9.3 8.3 Metriche e tracing 13
9.4 8.4 Strategia di test 13
9.5 Esercizio 13
9.6 Autoverifica 13

10 Modulo 9 — Progetto conclusivo: RAG agentico verificabile 13
10.1 Scenario 14
10.2 Architettura richiesta 14
10.3 Requisiti minimi di consegna 14
10.4 Criteri di valutazione (100 punti) 14
10.5 Discussione orale finale 14
10.6 Direzioni di approfondimento 15

11 Copioni originali per nove videolezioni 15

12 Videolezione 01 — Dentro un LLM: dal prompt al token 15

13 Videolezione 02 — Servire un modello locale con vLLM 15

14 Videolezione 03 — Costruire una Knowledge Base RAG valutabile 16

15 Videolezione 04 — Fine-tuning con LoRA e QLoRA 16

16 Videolezione 05 — Dal modello all'agente LangGraph 17

17 Videolezione 06 — Security e Human-in-the-Loop 17

18 Videolezione 07 — Prompt caching e prestazioni 18

19 Videolezione 08 — Troubleshooting di un sistema LLM 18

20 Videolezione 09 — Progetto finale e discussione tecnica 19

21 Calendario delle sei settimane 19
21.1 Rubrica di verifica individuale (0–4) 20
21.2 Domande trasversali per l'esame 20
21.3 Registro esperimenti 20
21.4 Risposte professionali alle cinque domande iniziali (scheletro) 20

22 Videotutorial verificati e riferimenti tecnici (consultati ottobre 2026) 20

1 Come usare questo manuale

Questo manuale accompagna un percorso tecnico di 36 ore articolato in nove moduli, con formazione scritta, riferimenti a videolezioni esistenti, copioni originali per screencast e laboratori Python scaricabili nel kit. **Non contiene file video riproducibili:** il pacchetto include script per registrarli e collegamenti alle risorse esterne.

Punto di partenza reale: Linux x86_64, vLLM 0.31.0 e Qwen/Qwen3-0.6B con inferenza REST dimostrata. **La GPU e la VRAM non sono ancora note**: non supponiamo che QLoRA o modelli più grandi siano eseguibili sulla macchina.

Metodo: teoria → video → laboratorio → misurazione → verifica. Tutti i documenti del corpus e tutti i dati di esempio sono fittizi. Non caricare dati di studenti o credenziali nei laboratori. I comandi devono essere verificati rispetto alle versioni effettivamente installate.

Per iniziare: leggere il file README.md nel pacchetto e installare le dipendenze del laboratorio applicativo in un ambiente Python separato da vLLM.

2 Modulo 1 — Fondamenti degli LLM

Durata: 3 ore. **Prerequisito:** Python base.

2.1 Obiettivi

Saper spiegare la generazione autoregressiva, i componenti del Transformer, i token, le finestre di contesto, il campionamento e le differenze fra pretraining, instruction tuning e inferenza.

2.2 1.1 Che cosa apprende un LLM

Un LLM autoregressivo riceve una sequenza di token e stima una distribuzione di probabilità per il token successivo: `P(token_t | token_1, ..., token_(t-1))`. La generazione ripete questo processo. Il modello **non** è un database fattuale: comprime regolarità statistiche e relazioni apprese in parametri. Può produrre affermazioni fluenti ma false, soprattutto su fatti assenti, mutati nel tempo o ambigui.

Il *tokenizer* trasforma il testo in identificatori numerici. I token non coincidono necessariamente con parole: una parola può contenere più token. Il costo dell'inferenza dipende dai token effettivi, non dai caratteri. `max_tokens` limita l'output e può troncarlo anche quando la risposta sarebbe semanticamente incompleta.

2.3 1.2 Dal testo alla previsione

Il flusso essenziale è: tokenizer → embedding di token → blocchi Transformer (self-attention + MLP con normalizzazione e connessioni residue) → proiezione sul vocabolario → logits → scelta del token. La *causal mask* impedisce a una posizione di leggere i token futuri durante l'inferenza autoregressiva.

La self-attention calcola, in forma semplificata, `Attention(Q,K,V) = softmax(Q K^T / sqrt(d_k)) V`. In un Transformer decoder, i vettori Q (query), K (key), V (value) derivano dalle rappresentazioni nascoste. Con più teste, il modello può apprendere relazioni differenti. Un numero elevato di parametri indica capacità, non garanzia di affidabilità.

2.4 1.3 Decodifica e parametri

`temperature=0` tende a privilegiare i token più probabili; valori maggiori favoriscono varietà. `top_p` limita il campionamento al sottoinsieme cumulativo di probabilità; `top_k`, ove disponibile, limita il numero di candidati. `max_tokens` è un limite di output. Il risultato può cambiare per versioni del modello, hardware, scheduler e determinismo numerico: la temperatura non è da sola una garanzia di riproducibilità.

Tre fasi da distinguere: **pretraining** (apprendimento generale), **fine-tuning/alignment** (adattamento del comportamento) e **inference** (uso dei pesi per rispondere). La modifica di un prompt non modifica i pesi del modello.

2.5 1.4 La risposta del nostro Qwen3

Nel test reale si vedeva `prompt_tokens=42`, `completion_tokens=256`, `finish_reason="length"`. È una risposta valida a livello API ma troncata; non prova che il modello abbia concluso. Si vedeva anche `<think> ... </think>` nel `content`: il comportamento è legato alla modalità di reasoning/template del modello e va gestito nel client, non scambiato per una seconda richiesta. Un modello piccolo può avere output in italiano poco corretti, anche se il serving funziona.

2.6 Esperimento

Lancia `python laboratori/01_inferenza_client.py`; confronta `max_tokens=64` e `max_tokens=400`; annota token input/output, `finish_reason`, durata, risposta e presenza o meno dei marcatori `<think>`.

2.7 Autoverifica

1. Perché 100 caratteri non sono sempre 100 token?
2. Che differenza esiste tra logits e probabilità?
3. Qual è il ruolo della causal mask?
4. Perché `finish_reason=length` non indica un errore dell'API?
5. Cosa cambia, e cosa **non** cambia, se aumenti `temperature`?

2.8 Video suggerito

https://www.deeplearning.ai/courses/how-transformer-llms-work

3 Modulo 2 — Local LLM & Model Serving

Durata: 4 ore.

3.1 2.1 Esecuzione locale: componenti

Un'installazione locale separa quattro livelli: (1) i pesi/tokenizer, (2) il runtime di inferenza, (3) il server HTTP, (4) il client. Nel tuo caso: `Qwen/Qwen3-0.6B` → vLLM → `/v1/chat/completions` → `curl` o client OpenAI. L'API compatibile consente di cambiare spesso il backend senza riscrivere tutta l'applicazione, ma non garantisce supporto identico a ogni parametro proprietario.

Transformers è una libreria generale per caricare e addestrare/eseguire modelli; **Ollama** semplifica packaging e uso locale; **vLLM** ottimizza il serving concorrente sfruttando gestione della KV cache, batching continuo e pianificazione delle richieste. **TGI** è un'altra opzione per il serving; la scelta dipende da hardware, modello, opzioni supportate, traffico, monitoring e deployment.

3.2 2.2 VRAM: non bastano i parametri

I pesi in BF16/FP16 richiedono in prima approssimazione 2 × N byte (con N parametri). Per 8 miliardi di parametri, circa 16 GB **solo di pesi**. A questi si aggiungono KV cache, buffer, activations, allocazioni del framework, prompt lunghi e richieste concorrenti. Quantizzare i pesi a 8 o 4 bit riduce tale componente, con overhead e possibili compromessi in qualità; non elimina automaticamente la KV cache.

Misure richieste: `nvidia-smi`, VRAM libera/occupata, throughput token/s, TTFT (time-to-first-token), latenza end-to-end, TPOT (time per output token), P50/P95/P99, error rate e concurrency. Distinguere throughput aggregato da latenza individuale: l'aumento di throughput può peggiorare P95.

3.3 2.3 Prefill, decode, KV cache, batching

Prefill elabora il prompt e popola Key/Value per i token del contesto. **Decode** genera token successivi riutilizzando le rappresentazioni memorizzate. La **KV cache** evita di ricalcolare tutta la storia a ogni token, ma consuma memoria GPU. Il **continuous batching** permette di aggiungere/togliere richieste senza aspettare che l'intero gruppo termini; ciò migliora l'uso della GPU. **PagedAttention** gestisce blocchi di memoria per le rappresentazioni KV in modo più efficiente.

Le impostazioni di scheduling, parallelismo e caching variano con la versione di vLLM. Usare sempre `vllm serve --help` e `vllm --version` sul server reale prima di copiare opzioni da una guida esterna.

3.4 2.4 Laboratorio consigliato

```
# Nel terminale server (ambiente vLLM preesistente)
vllm serve Qwen/Qwen3-0.6B --host 127.0.0.1 --port 8000 \
  --dtype auto --max-model-len 2048 --gpu-memory-utilization 0.75

# Nel terminale client
curl -s http://127.0.0.1:8000/v1/models
python laboratori/02_benchmark_concorrenza.py --concurrency 1 --requests 12
python laboratori/02_benchmark_concorrenza.py --concurrency 5 --requests 20
```

Confronta la latenza mediana, la P95, il numero di completamenti riusciti e il throughput approssimato. Per un benchmark più rigoroso verifica `vllm bench serve --help` e fissa la lunghezza dell'input e output.

3.5 2.5 Produzione

Esporre `127.0.0.1` consente accesso locale; usare `0.0.0.0` rende la porta potenzialmente raggiungibile dalla rete. Prima di esporre un server LLM: autenticazione, TLS in reverse proxy, rate limiting, limiti per utente, log privi di contenuti sensibili, timeouts, quote e monitoraggio della GPU. Docker non equivale a isolamento di sicurezza completo. In Kubernetes: Deployment, Service, resource limits GPU, readiness probe, autoscaling realistico e ingress autenticato; non pubblicare direttamente un server senza protezioni.

3.6 Autoverifica

1. Come distingui KV cache e pesi?
2. Perché 50 richieste concorrenti non implicano 50 modelli in VRAM?
3. Cosa misureresti per scegliere tra Ollama e vLLM?

4. Perché una GPU con 16 GB non garantisce che un 8B FP16 funzioni con prompt lunghi?
5. Che cosa cambia tra `host 127.0.0.1` e `host 0.0.0.0`?

3.7 Video suggeriti

https://www.deeplearning.ai/courses/fast-and-efficient-llm-inference-with-vllm https://www.deeplearning.ai/c
serving-llms

4 Modulo 3 — Retrieval-Augmented Generation e Knowledge Base

Durata: 5 ore.

4.1 3.1 Che problema risolve

Un modello generativo può non conoscere i dettagli aggiornati di un regolamento o di un catalogo. Il RAG recupera documenti esterni pertinenti alla domanda e li inserisce nel contesto della generazione. La fonte *resta esterna* ai pesi: si può aggiornare la KB senza riaddestrare l'LLM. Ciò non garantisce automaticamente veridicità: recupero sbagliato, citazioni inventate, documenti obsoleti o istruzioni malevole possono compromettere le risposte.

4.2 3.2 Pipeline di ingestione

`documenti → estrazione del testo → pulizia/versionamento → chunking → embeddings → indicizzazione`. Ogni chunk dovrebbe conservare almeno `document_id`, URL/percorso, titolo, data, versione, permessi e intervallo/posizione d'origine. Il chunking a caratteri è semplice ma non allineato ai token: in produzione provare segmentazione semantica o per sezione, overlap controllato e misure di recall.

Gli **embedding** trasformano testi o domande in vettori. La similarità coseno misura la vicinanza angolare tra vettori normalizzati. Il modello di embedding può essere diverso dall'LLM generativo; lo deve essere quasi sempre, in un RAG classico. Per contenuti multilingue, scegliere un embedding realmente multilingue e valutarlo nella lingua degli utenti.

Il **vector DB** memorizza vettori, metadati e indici efficienti. Esempi: Chroma (locale), Qdrant, FAISS (libreria ANN, non un DB completo), pgvector. La ricerca può essere densa (semantic embeddings), lessicale (BM25) o **ibrida** con fusione dei risultati. Un **reranker** riordina i documenti candidati con una stima più costosa ma potenzialmente più accurata.

4.3 3.3 Query time

`domanda → embedding della domanda → retrieve top-k → (eventuale BM25+fusion) → rerank → contesto con source IDs → prompt → risposta → citazioni`. Il prompt va scritto per utilizzare i documenti come **dati non attendibili**: un documento non può ridefinire il ruolo dell'assistente o autorizzare tool.

Controllare la lunghezza dei chunk selezionati: troppo testo peggiora costo/latency e può diluire le evidenze. Non confondere rilevanza dei documenti con fedeltà della risposta. Un buon retrieval può essere vanificato da una generazione sbagliata.

4.4 3.4 Evaluation: una condizione per parlare di produzione

Separare due categorie: - **Retrieval**: Recall@k/HitRate@k, MRR, nDCG, copertura di domande e permessi. - **Generazione**: faithfulness/groundedness, correctness rispetto a gold answers, completezza, qualità delle citazioni, percentuale di astensione quando le fonti sono insufficienti, latenza e costo.

Creare almeno 20-50 casi di test con: domanda, risposta attesa, source ID atteso, difficoltà, categoria, lingua e comportamento richiesto in assenza di fonti. Tenere un insieme separato per modifiche e regressioni. Una valutazione automatica con un altro LLM è utile ma non sostituisce controlli umani e criteri espliciti.

4.5 3.5 Laboratorio

Il dataset `dati/kb_demo.json` è **fittizio**, non contiene regolamenti reali. I programmi `03_indicizza.py` e `03_rag.py` creano una KB Chroma persistente, usano un modello sentence-transformers multilingue e interrogano l'LLM locale tramite API vLLM. `03_eval_retrieval.py` calcola HitRate@k sul dataset campione. Dopo il baseline: prova chunk diversi, top-k differenti, documenti non pertinenti e una domanda senza fonte, monitorando le variazioni.

4.6 Autoverifica

1. Qual è la differenza tra indice vettoriale e KB completa?
2. Perché un reranker non sostituisce l'embedding model?
3. Cosa misura Recall@5?
4. Perché non basta testare soltanto la qualità della risposta finale?
5. Come gestisci cancellazione/versione/permessi dei documenti?

4.7 Video suggerito

https://www.deeplearning.ai/courses/building-evaluating-advanced-rag

5 Modulo 4 — Fine-Tuning, LoRA e QLoRA

Durata: 5 ore.

5.1 4.1 Quando scegliere il fine-tuning

Il fine-tuning adatta i parametri (o una loro piccola componente aggiuntiva) a esempi supervisionati. È indicato quando bisogna rendere stabile uno **stile**, un **formato strutturato**, una **tassonomia**, un comportamento di tool calling, o migliorare su un compito ricorrente. RAG è spesso preferibile per fonti aggiornabili, verificabili e citabili. I due approcci possono convivere: fine-tuning per comportamento, RAG per conoscenza accessibile e aggiornata.

Non usare fine-tuning per correggere un indice RAG difettoso o per mettere nel modello segreti e dati personali non autorizzati. Prima di addestrare, confronta un *baseline* che usa buon prompting e RAG.

5.2 4.2 Tecniche

Full fine-tuning modifica tutti i pesi e può richiedere hardware molto costoso. **LoRA** congela i pesi base e apprende matrici a basso rango che approssimano l'adattamento di alcune trasformazioni lineari: $W' = W + \Delta W$, con $\Delta W \approx B\,A$. Parametri importanti: `r`, `lora_alpha`, `lora_dropout`,

`target_modules`, learning rate, steps/epochs. Con **QLoRA**, il modello base è caricato in forma quantizzata, tipicamente a 4 bit, mentre gli adapter sono allenati con precisione appropriata: ciò riduce memoria, ma non annulla i costi di attivazioni e optimizer.

SFT usa esempi input → output e una loss di previsione token; DPO e tecniche di preference optimization servono ad altri obiettivi. Non confondere un adapter LoRA con un intero modello indipendente: per inferenza serve base model + adapter, salvo fusione compatibile.

5.3 4.3 Dataset e qualità

Ogni esempio deve rappresentare il task desiderato, essere revisionato, non includere risposte contraddittorie o PII non autorizzati e rispettare la chat template del modello. Separare train/dev/test **prima** di costruire prompt o riutilizzare esempi. Un microdataset sintetico serve per verificare la pipeline, non per dimostrare un aumento di qualità.

Misure prima/dopo: exact match o F1 dove appropriato, validità JSON, accuratezza della classificazione, allucinazioni, safety regressions, qualità della lingua, costo inferenziale. Annotare seed, checkpoint, versione librerie, dataset, hyperparameters e benchmark riproducibili.

5.4 4.4 Laboratorio

`laboratori/04_finetuning_lora.py` propone una sessione **dimostrativa** con Qwen3-0.6B, TRL e PEFT, dataset fittizio e LoRA. L'esecuzione richiede GPU adeguata e un ambiente training isolato. Prima interrompere vLLM, che può già occupare molta VRAM. Salvare gli adapter; poi confrontare alcuni prompt non visti con il modello di partenza. Non considerare un training terminato con successo come prova di un miglioramento senza evaluation indipendente.

5.5 Autoverifica

1. Cosa viene aggiornato in LoRA?
2. In QLoRA che cosa è quantizzato?
3. Quando il RAG è più corretto rispetto al fine-tuning?
4. Cosa misura un test set separato?
5. Perché un dataset di 12 esempi non dimostra una competenza di produzione?

5.6 Video suggerito

https://www.deeplearning.ai/courses/finetuning-large-language-models ## Riferimento tecnico https://huggingface.co/docs/trl/peft_integration

6 Modulo 5 — Agentic AI, tool calling, ReAct, LangGraph e MCP

Durata: 6 ore.

6.1 5.1 Definizioni da non confondere

Un **workflow** è una sequenza di operazioni con regole definite dal programma. Un **agente** delega alcune decisioni al modello, per esempio quale strumento chiamare o quale sotto-obiettivo perseguire. Un **tool** è una funzione esterna (ricerca, interrogazione SQL, RAG, calcolo, API). Il tool calling richiede *due passaggi*: il modello propone una chiamata strutturata e il codice applicativo la valida/esegue; il modello non possiede automaticamente permessi di sistema.

Il paradigma **ReAct** alterna elaborazione del problema, selezione di azioni, osservazione dei risultati e iterazione. Un **planner-executor** separa la pianificazione dall'esecuzione; un **router** classifica la richiesta e seleziona un ramo; un **critic/editor** controlla la bozza e decide se correggerla. La presenza di più nodi non garantisce un miglioramento: aumenta latenza, costo e punti di errore. Limitare passi e cicli.

6.2 5.2 Stato e grafo

In LangGraph i nodi sono funzioni che ricevono uno stato e restituiscono aggiornamenti; gli edge definiscono i percorsi, anche condizionali. Lo **State** descrive i dati condivisi. Un *checkpointer* salva lo stato per ripresa, HITL e debugging; uno store persistente può ospitare memoria a lungo termine. Memoria conversazionale, cache e KB hanno scopi differenti.

Per una pipeline di analisi di mercato: `intent_router → researcher → analyzer → writer → critic → editor → final_answer`. Per ogni nodo specificare **input, trasformazione, output, errori**. Il router può classificare `technology`, `economics`, `jobs`, `sport`, `scientific_research`; il researcher può usare Tavily o una KB. Le dipendenze iniettate (`IntentRouterNode(OllamaRouter(settings))`) sono una **relazione di composizione/delega**, non un edge di esecuzione del grafo: UML delle classi e grafo dati vanno rappresentati separatamente.

6.3 5.3 Tool contracts e structured outputs

Definire schema dei parametri, campi obbligatori, limiti temporali, whitelist di domini/risorse, ritentativi idempotenti e forma dei risultati. Un tool dovrebbe restituire errori leggibili dal codice, non contenuto HTML non filtrato. Quando il modello produce JSON, validarlo con `pydantic`/JSON schema. Evitare di fidarsi di un campo `approved` inventato dal modello.

6.4 5.4 MCP (Model Context Protocol)

MCP è uno standard che permette a client e server di esporre strumenti e risorse attraverso un'interfaccia comune. Non equivale a un framework multi-agent e non sostituisce autorizzazione, autenticazione o sandbox. Occorre trattare tool description e risposte esterne come superfici non attendibili, stabilire permessi granulari e controllare le operazioni con effetti esterni.

6.5 5.5 Agentic RAG

In un RAG classico, il retriever spesso viene chiamato per ogni domanda. In un **Agentic RAG**, l'agente può scegliere se interrogare una KB, fare un'altra ricerca, chiedere chiarimenti o confrontare fonti, ma questa autonomia deve essere misurata. Valutare: tool selection accuracy, task success, numero di step, qualità delle prove, costo e fallimenti per tipo.

6.6 5.6 Laboratorio

`laboratori/05_agent_langgraph.py` realizza un piccolo grafo locale con tre funzioni: ricerca su dati fittizi, stesura tramite vLLM e verifica deterministica delle fonti. Espandilo progressivamente fino alla pipeline router/research/analyzer/writer/critic/editor. Non iniziare da sette agenti indipendenti: prima dimostrare che una singola pipeline a tre nodi produca risposte controllabili.

6.7 Autoverifica

1. Qual è la differenza tra tool calling e accesso effettivo a uno strumento?
2. In LangGraph che cosa distingue un nodo da un edge?

3. Qual è il ruolo dello stato e di un checkpointer?
4. In cosa differiscono ReAct, router e planner-executor?
5. Come eviti cicli illimitati, tool ripetuti e costi imprevedibili?

6.8 Videotutorial

https://academy.langchain.com/courses/intro-to-langgraph https://www.deeplearning.ai/courses/ai-agents-in-langgraph https://huggingface.co/learn/agents-course/en/unit3/agentic-rag/agentic-rag

7 Modulo 6 — Platform, Security e Human-in-the-Loop

Durata: 4 ore.

7.1 6.1 Piattaforma: dove si trova ogni componente

Separare **control plane** (identità, configurazioni, orchestrazione), **data plane** (inferenza, retrieval, tool execution) e **observability plane** (log, metriche, trace). In locale, vLLM serve il modello e il backend Python gestisce retrieval e agenti. In cloud, si possono utilizzare Bedrock per modelli gestiti o GPU self-hosted su EC2/EKS/ECS EC2; evitare l'errore concettuale per cui ECR sarebbe un orchestratore: ECR conserva immagini, ECS/EKS gestiscono esecuzione, IAM gestisce autorizzazioni.

Un percorso di deploy tipico per modelli self-hosted: Git → CI/test → image build → ECR → ECS su EC2 o EKS con nodi GPU → ALB/ingress → API. Bedrock sostituisce il componente model serving ma non tutta l'applicazione. Specificare sempre region, model access, IAM role/STS identity, secret management, timeouts e retry policy.

7.2 6.2 Fiducia tra componenti

Il **prompt injection** sfrutta istruzioni malevole in input non attendibili: email, pagine web, documenti recuperati dal RAG, output di un tool. La difesa non è semplicemente "dire al modello di ignorare". Serve una barriera **nel codice**: classificazione dei dati, access control verificato sul server, validazione dei parametri, permission boundary per tenant, URL/path allowlist, tool limitati in scrittura, output escaping, auditing e test avversariali.

Il modello non deve poter invocare direttamente comandi shell arbitrari, cancellare risorse o mandare email senza controllo. Non includere chiavi API, dati privati o credenziali in prompt, log o dataset. I documenti della KB possono essere attendibili come fonte di fatti ma **non** come fonte di istruzioni per l'agente.

7.3 6.3 Human-in-the-Loop (HITL)

HITL significa sospendere l'esecuzione **prima** di un'azione sensibile: il sistema crea una richiesta di revisione, salva lo stato, espone all'operatore il piano e attende una decisione. Esiti tipici: approva, modifica, rifiuta. Una richiesta approvata può riprendere **solo** se l'identità e i permessi dell'approvatore sono stati verificati sul backend.

In LangGraph, `interrupt(...)` consente una pausa nel grafo e `Command(resume=...)` la ripresa. La persistenza non è opzionale in uno scenario reale: un checkpointer in memoria serve solo per demo e non resiste ai riavvii. Attenzione agli effetti collaterali eseguiti **prima** dell'interrupt: il nodo potrebbe essere rieseguito dopo la ripresa. Usare azioni idempotenti e spostare gli effetti dopo l'approvazione.

7.4 6.4 Esercizio pratico

`laboratori/06_hitl.py` realizza una revisione umana su una bozza di messaggio, senza inviarlo realmente. Esegui il caso approvato, poi quello rifiutato. Spiega perché l'approvazione simulata da input locale non è sufficiente come autenticazione in un servizio multi-utente.

7.5 Autoverifica

1. Cosa succede se un PDF indicizzato scrive "ignora tutte le regole"?
2. Perché un modello non può approvare da solo una sua operazione sensibile?
3. Come distingui IAM role, credenziali AWS e accesso al modello Bedrock?
4. Perché un checkpointer persistente serve per HITL robusto?
5. Dove metti i controlli di autorizzazione per uno strumento di cancellazione?

7.6 Videotutorial

https://www.deeplearning.ai/courses/safe-and-reliable-ai-via-guardrails https://academy.langchain.com/cours to-langgraph ## Riferimento di sicurezza https://genai.owasp.org/llmrisk/llm01-prompt-injection/

8 Modulo 7 — Prompt Caching, Prefix Caching, KV Cache e costi

Durata: 3 ore.

8.1 7.1 Tre cache diverse

1. **KV cache (inference-time state)**: Key/Value per i token già processati durante la generazione corrente. Evita di ricalcolare la storia a ogni token, ma occupa VRAM.
2. **Prefix/prompt caching**: riusa parte della computazione di prefill tra richieste diverse che condividono lo stesso **prefisso di token**. Alcuni fornitori gestiti fatturano in modo specifico cache write/cache read; le regole dipendono dal modello.
3. **Semantic response caching**: può restituire una risposta memorizzata per domande simili, evitando del tutto un'inferenza. È rischiosa se i dati sono personalizzati, cambiano rapidamente o richiedono citazioni contestuali.

Il *caching di risultati dei tool* è ancora un quarto problema: bisogna conservare validità temporale, permessi e versione dei dati. Non usare un risultato cached tra tenant diversi senza una politica di isolamento ben progettata.

8.2 7.2 Automatic Prefix Caching in vLLM

vLLM può riutilizzare blocchi di KV cache già calcolati quando due richieste hanno un prefisso comune; su vLLM 0.31.0 l'opzione `enable_prefix_caching` esiste. Non dedurre che un hit si verifichi ogni volta: dimensione del prefisso, disponibilità nella cache, politiche di eviction e tokenizzazione influiscono. Il vantaggio riguarda **prefill**, non accelera direttamente la generazione dei token nuovi.

Per il laboratorio: mantieni identica una lunga parte statica del system prompt e modifica solo la domanda finale. Confronta TTFT con caching attivo/inattivo, ripeti più volte e misura mediane; un unico esempio è solo indicativo. Verifica l'effettiva opzione con `vllm serve --help | grep -i prefix`.

8.3 7.3 Bedrock e provider gestiti

Amazon Bedrock supporta, per modelli idonei, **implicit caching** e **explicit caching** tramite checkpoint o controlli di cache specifici della API/modello. Il prefisso statico va collocato prima del contenuto dinamico. La presenza di un checkpoint non garantisce cache hit; verifica i campi di usage della risposta. Limiti minimi di token, TTL, region, pricing e endpoint supportati cambiano in base al modello: consulta sempre la documentazione prima di implementare.

8.4 7.4 Errori concettuali frequenti

- "La KV cache mi salva i prompt su disco": falso come descrizione generale.
- "Prefix caching significa semantic caching": falso, il primo tipicamente richiede prefissi/token allineati.
- "Il caching risolve le allucinazioni": falso.
- "Un cache hit è sempre economicamente vantaggioso": non necessariamente, dipende da costi di scrittura, lettura, TTL e riutilizzi.

8.5 7.5 Laboratorio

`laboratori/07_prefix_cache_lab.py` invia più richieste con prefisso fisso e domande variabili, misurando tempi end-to-end. Per una misura vera del TTFT, leggere eventi `stream=True` anziché il tempo totale. Per l'esperimento rigoroso fare due riavvii separati del server, uno con caching attivo e uno disattivato, e verificare metriche del server.

8.6 Autoverifica

1. Qual è la distinzione tecnica tra KV cache e prefix caching?
2. Come massimizzi il riuso di un system prompt condiviso?
3. Perché cambiare la parte iniziale del prompt produce cache miss?
4. Come controlli che Bedrock abbia effettivamente riutilizzato token?
5. Quali problemi crea la semantic cache in un chatbot multi-tenant?

8.7 Videotutorial

https://www.deeplearning.ai/courses/building-toward-computer-use-with-anthropic https://www.deeplearning
and-efficient-llm-inference-with-vllm ## Riferimenti https://docs.vllm.ai/en/v0.31.0/features/automatic_prefi
https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html

9 Modulo 8 — Troubleshooting, osservabilità e quality assurance

Durata: 3 ore.

9.1 8.1 Diagnostica per livelli

Un errore end-to-end va localizzato per componente, senza modificare tutto insieme: 1. **Host/GPU**: `uname -m`, `nvidia-smi`, driver, VRAM disponibile, processi residenti. 2. **Runtime**: ambiente Python attivo, `which vllm`, `vllm --version`, PyTorch/CUDA compatibili. 3. **Model loading**: identificativo e accessibilità del modello, tokenizer/chat template, dimensione della context window. 4. **Serving HTTP**: bind host, porta occupata, readiness endpoint, request schema, errori 400/401/404/429/500. 5. **Applicazione**: Pydantic/JSON parsing, prompt injection, fallimenti

di tool, timeout/retry. 6. **Qualità**: retrieval errato, chunk non pertinenti, allucinazioni, citazioni difettose, regressioni del modello.

9.2 8.2 Problemi incontrati nell'esercitazione

`vllm download-kernels` non riconosciuto: **subcommand assente** in quella CLI. Prima di usare esempi da Internet, verificare `vllm --help`, non reinstallare alla cieca. `finish_reason=length`: risposta interrotta al limite `max_tokens`; aumentare solo dopo aver controllato VRAM, prompt length e comportamento del modello. Marcatori `<think>` in `message.content`: comportamento legato a Qwen3/template/parametri, non necessariamente un crash; usare il parametro per disabilitare thinking quando supportato.

`CUDA out of memory`: verificare disponibilità effettiva, `gpu_memory_utilization`, `max_model_len`, quantizzazione, batch/concurrency e processi concorrenti. Cambiare i parametri uno per volta e salvare benchmark. `Unable to locate credentials` con boto3: verificare identity/credential provider chain **nell'ambiente in cui gira il processo**, non solo sull'host. `Access to model ... not available`: può essere questione di permessi, region, entitlement o tipo di account; verificare con i servizi AWS e gli identificativi corretti del modello.

9.3 8.3 Metriche e tracing

Registrare request ID e correlation ID tra frontend, backend, agente, vector DB e model server. Per ogni chiamata: modello, versione, token input/output, dimensione contesto, numero di retrieval, query rewriting, chiamate tool, errori, latenza, TTFT, cache hit (se disponibile). Evitare di salvare prompt privati nei log senza basi legali e protezioni. Le metriche aggregate non sostituiscono trace diagnostiche con cause/contesto.

9.4 8.4 Strategia di test

Unit test di parser, routing e tool; **integration test** di RAG/model server; **eval offline** con dataset versionato; **load test** con concorrenza e rate limit; **security test** con prompt injection; **canary** e rollback in produzione. Non trattare un singolo output convincente come misura statistica.

9.5 Esercizio

Lancia `laboratori/08_diagnostica.py`; simula una porta errata per verificare la gestione degli errori; ripeti con il server attivo. Crea una scheda incidente con "sintomo, ipotesi, prova, esito, correzione, regressione".

9.6 Autoverifica

1. Come distingui un errore CUDA da un errore applicativo HTTP?
2. Quali evidenze raccogli per un errore 500 intermittente?
3. Che differenza c'è tra log, metriche e trace?
4. Come dimostri che una modifica al retrieval ha migliorato la qualità?
5. Perché la procedura di rollback fa parte del troubleshooting?

10 Modulo 9 — Progetto conclusivo: RAG agentico verificabile

Durata: 3 ore per l'integrazione base; estensioni facoltative 8-12 ore.

10.1 Scenario

Progettare un assistente che risponde a domande su una knowledge base dimostrativa, distingue quesiti documentali da domande generali, cita le fonti, registra i risultati e sottopone alla revisione umana ogni azione sensibile. L'esempio deve usare solo i documenti **sintetici** in `dati/`, non dati reali di studenti.

10.2 Architettura richiesta

`Utente → API backend → Intent Router → [KB Retriever / General LLM] → Risposta con fonti → Critic/Eval → Output`. Se la richiesta necessita una modifica o un invio verso sistemi esterni, instradarla a `HITL approval → tool con autorizzazione verificata → audit`. Il sistema LLM locale resta un backend di inferenza, non decide da solo l'accesso alle informazioni o ai tool.

10.3 Requisiti minimi di consegna

1. Un endpoint Python o script CLI interrogabile che usa il modello Qwen in vLLM.
2. Un archivio di documenti indicizzato con un embedding model differente dal modello generativo.
3. Retrieval top-k con ID delle fonti e comportamento di astensione quando la documentazione non basta.
4. Un workflow LangGraph con stato esplicito, un ramo condizionale e un limite massimo di ripetizioni.
5. Un esempio HITL di approvazione/rifiuto senza eseguire azioni irreversibili.
6. Un set di almeno 20 domande di test; 5 per ciascuna categoria: semplici, multi-hop, senza risposta, tentativi di prompt injection.
7. Report con HitRate@k, almeno una misura di groundedness giudicata manualmente, latenza mediana/P95, error rate e limite di memoria usata.

10.4 Criteri di valutazione (100 punti)

Dimensione	Peso
Correttezza tecnica e riproducibilità	20
Retrieval ed evidence/citazioni	20
Esecuzione agentica controllata	15
Fine-tuning: progettazione e motivazione (anche senza GPU adeguata)	10
Sicurezza, autorizzazioni e HITL	15
Tracing, metriche, test e rollback	15
Documentazione delle decisioni e limitazioni	5

10.5 Discussione orale finale

Preparare risposte sintetiche a cinque domande: quale modello open-weight hai eseguito? Su quale GPU/VRAM? Hai svolto LoRA/QLoRA personalmente? Hai realizzato e valutato un RAG realmente utilizzato? Quale inference server hai configurato? Distinguere esperienza laboratoriale, sperimentale e produzione reale: non attribuire a un laboratorio didattico risultati di produzione.

10.6 Direzioni di approfondimento

Multi-agent evaluation; router adattivi; speculative decoding; agent memory; guardrail e test red-team; Model Context Protocol e autorizzazione dei tool; A/B testing; deployment EKS GPU; telemetry OpenTelemetry; semantic cache con invalidazione e isolamento per tenant.

11 Copioni originali per nove videolezioni

Ogni copione può essere letto come traccia di una screencast in italiano. I tempi sono indicativi, non file video già prodotti.

12 Videolezione 01 — Dentro un LLM: dal prompt al token

Durata suggerita: 8-10 minuti. **Formato:** screencast 16:9, terminale e slide. **Obiettivo:** distinguere token, Transformer, sampling e fine del completamento.

00:00-01:00 — Apertura, inquadratura del terminale. Voce: «Un LLM non cerca una frase pronta in un archivio. Data una sequenza di token, stima quali token possano seguire. Per questo è in grado di generare testo, ma può anche costruire frasi fluenti non corrette. Oggi useremo un modello che abbiamo già eseguito in locale: Qwen3-0.6B, accessibile tramite vLLM.»

01:00-03:00 — Slide "tokenizer → embedding → Transformer → logits → token". Voce: «Il testo passa prima dal tokenizer, che lo spezza in unità numeriche. Le rappresentazioni vengono trasformate dai blocchi Transformer. L'attenzione permette a ogni posizione di combinare informazioni dal contesto disponibile, nel rispetto della causal mask. Infine il modello produce punteggi per i possibili token successivi: i logits.»

03:00-05:00 — Schermo diviso: temperatura e `max_tokens`. Voce: «I parametri di campionamento non cambiano ciò che è stato appreso. Cambiano il modo di scegliere il prossimo token. Con `max_tokens=64` possiamo troncare una spiegazione corretta a metà frase. Il nostro precedente output mostrava proprio `finish_reason=length`: il server funzionava, era soltanto finito il budget di generazione.»

05:00-08:00 — Demo. Mostra `curl localhost:8000/v1/models`; apri `laboratori/01_inferenza_client` ed eseguilo. Evidenzia `usage.prompt_tokens`, `usage.completion_tokens` e `finish_reason`. Spiega che il prefisso `<think>` apparso in un esperimento dipende dalla modalità del modello e dal template.

08:00-10:00 — Chiusura e verifica. Domanda orale: «Perché 256 token in output non significano 256 parole? Che cosa posso inferire da una risposta troncata?». Assegna esercizio: cambiare un solo parametro e annotare la differenza, senza attribuire variazioni al caso sbagliato.

Risorsa esterna: https://www.deeplearning.ai/courses/how-transformer-llms-work

13 Videolezione 02 — Servire un modello locale con vLLM

Durata suggerita: 12-15 minuti. **Riprese:** desktop Linux, `nvidia-smi`, terminali affiancati e un semplice grafico concorrenza/latency.

00:00-02:00 — Il sistema reale. Voce: «Abbiamo installato vLLM in un ambiente virtuale Linux e verificato una risposta di Qwen3-0.6B. Mostro dove si trovano il modello, il server e il client. Il nome della GPU non va indovinato: lo ricaviamo dai dati di `nvidia-smi`. Il confronto con 7B, 8B e 32B dipende da VRAM e quantizzazione.»

02:00-04:00 — Memoria e throughput. Mostra due rettangoli “pesi” e “KV cache”. Voce: «I pesi restano caricati, mentre la KV cache cresce con prompt, contesto e richieste. La memoria richiesta non è il semplice numero di parametri. Aumentare la lunghezza massima del contesto consuma capacità che potrebbe servire alla concorrenza.»

04:00-06:00 — Prefill e decode. Voce: «Il prefill processa il contesto; il decode genera un token alla volta. Continuous batching permette di far entrare nuove richieste nel motore mentre altre sono già in fase di generazione. PagedAttention riduce sprechi nella gestione della cache.»

06:00-11:00 — Laboratorio reale. Esegui `vllm --version`, `curl /v1/models`, quindi `python laboratori/02_benchmark_concorrenza.py --concurrency 1 --requests 12` e `--concurrency 5 --requests 20`. Annota su schermo mediane e P95 senza inventare valori. Spiega perché l’aumento di richieste può migliorare throughput ma peggiorare latenza.

11:00-15:00 — Mini colloquio. «Quale server hai usato? Quale modello? Su quale GPU? Quali metriche hai misurato?». Distinguere un test sul proprio PC dal serving in produzione.

Risorsa: https://www.deeplearning.ai/courses/fast-and-efficient-llm-inference-with-vllm

14 Videolezione 03 — Costruire una Knowledge Base RAG valutabile

Durata suggerita: 12-15 minuti. **Riprese:** slide della pipeline, editor Python, terminale, risultati di HitRate@k.

00:00-02:00 — Il problema. «Un assistente che risponde a domande su documenti aggiornati non deve affidarsi soltanto alle conoscenze apprese nel pretraining. RAG aggiunge retrieval prima della generazione. Le fonti rimangono in un archivio modificabile.»

02:00-05:00 — Ingestione. Mostra i sei documenti sintetici in `dati/kb_demo.json`. «Dividiamo un corpus in unità recuperabili, generiamo vettori con un embedding multilingue e li indicizziamo con Chroma. Un database vettoriale non equivale alla KB completa: metadati, permessi, data e versioni contano quanto i vettori.»

05:00-08:00 — Retrieval e risposta. Esegui `python 03_indicizza.py` e `python 03_rag.py "Chi approva le comunicazioni esterne?"`. Indica i tre risultati ordinati per distanza. «Passiamo al modello solo i chunk recuperati, ma sono dati: eventuali istruzioni dentro un PDF non devono diventare comandi dell’agente. Le fonti stampate dal codice non certificano da sole l’accuratezza di ogni frase.»

08:00-12:00 — Evaluation. Esegui `python 03_eval_retrieval.py --k 1` e poi `--k 3`. «HitRate@k misura se il documento atteso è presente tra i candidati. Per valutare l’output, dobbiamo controllare groundedness, completezza, astensione e citazioni. Il piccolo dataset serve per dimostrare il metodo, non per certificare il prodotto.»

12:00-15:00 — Compito. Aggiungi un documento irrilevante e ripeti i test; poi prova una domanda senza fonte. Segna cosa è cambiato e perché.

Risorsa: https://www.deeplearning.ai/courses/building-evaluating-advanced-rag

15 Videolezione 04 — Fine-tuning con LoRA e QLoRA

Durata suggerita: 12-15 minuti. **Riprese:** diagramma dei pesi congelati/adapters, editor e training log se l’hardware lo consente.

00:00-03:00 — La scelta progettuale. «Prima di addestrare un LLM domandiamoci se vogliamo cambiare le fonti informative oppure il comportamento. Per conoscenza aggiornata e citabile,

spesso vince RAG. Se il modello sbaglia sistematicamente un formato o un task, può essere utile il fine-tuning.»

03:00-06:00 — Tre tecniche. Mostra W, A e B su slide. «Full fine-tuning aggiorna i pesi del modello. LoRA impara una correzione di rango ridotto mantenendo fissi i pesi base. QLoRA carica il modello base quantizzato, riducendo la memoria per gli stessi adapter. Più piccolo non equivale a gratuito: servono RAM, attivazioni, ottimizzatore e verifiche.»

06:00-09:00 — Dataset. Apri `04_finetuning_lora.py`: «Qui ci sono esempi sintetici di domande e risposte. Lo scopo è imparare la struttura del dataset, la chat template, il rank `r` e i target modules. Dieci esempi di train e due di validazione non misurano un miglioramento reale.»

09:00-12:00 — Laboratorio opzionale sulla GPU. Prima di eseguire: mostra `nvidia-smi`, spegni vLLM e attiva ambiente training. Esegui solo se CUDA e VRAM sono compatibili. Osserva loss e adapter salvato, evitando conclusioni assolute. Se la macchina non è adeguata, presenta il codice e i prerequisiti senza simulare un training eseguito.

12:00-15:00 — Autoverifica. «Ho davvero aggiornato pesi o soltanto cambiato un prompt? Posso dimostrare un miglioramento su domande non viste? Che cosa devo dichiarare a un colloquio tecnico?».

Risorsa: https://www.deeplearning.ai/courses/finetuning-large-language-models

16 Videolezione 05 — Dal modello all'agente LangGraph

Durata suggerita: 14-18 minuti. **Riprese:** diagramma di stato, codice del grafo e trace dei nodi.

00:00-03:00 — Agente vs workflow. «Un workflow chiama funzioni secondo il programma; un agente lascia alcune decisioni al modello. In entrambi i casi, i permessi appartengono al software, non alla frase prodotta dall'LLM. Tool calling è una richiesta strutturata, eseguita solo dopo validazione.»

03:00-06:00 — Il grafo. Disegna `retrieve → draft → verify → {done/retry}`. «Ogni nodo legge lo stato e ne restituisce aggiornamenti. Un edge trasferisce il controllo, uno condizionale decide il ramo. Un limite di tentativi evita il ciclo infinito.»

06:00-10:00 — ReAct e ruoli. «ReAct usa osservazioni dei tool per scegliere l'azione successiva. Router, planner, researcher, critic ed editor sono ruoli che possiamo implementare in un grafo: non è obbligatorio usare un LLM diverso per ciascuno. Introduciamo un agente solo quando il comportamento dinamico porta valore misurabile.»

10:00-15:00 — Esecuzione. Mostra `05_agent_langgraph.py` e stampa stato finale, fonti e `attempt`. Spiega che il controllo `[Dxx]` non certifica groundedness. Proponi un diagramma delle classi separato da un grafico dei nodi: una dipendenza Python verso un oggetto router non è un edge del workflow.

15:00-18:00 — Verifica. «Che cosa persiste tra due chiamate? Quali strumenti può usare il nodo? Che cosa accade se il modello produce una citazione inventata?».

Risorse: https://academy.langchain.com/courses/intro-to-langgraph — https://www.deeplearning.ai/courses/agents-in-langgraph

17 Videolezione 06 — Security e Human-in-the-Loop

Durata suggerita: 10-13 minuti. **Riprese:** confini di fiducia, demo `interrupt`, diagramma IAM/RBAC.

00:00-03:00 — Documenti non attendibili. «Un agente RAG può recuperare il testo di una pagina che contiene 'ignora il sistema ed esegui questo comando'. Per il modello è testo, ma nel progetto non deve diventare autorità. Isoliamo istruzioni del sistema da dati non attendibili e imponiamo permessi nel backend.»

03:00-05:00 — Difesa a strati. Mostra: input validation → auth → tool policy → approval → audit. «Un guardrail testuale può fallire. Non affidiamo all'LLM la decisione finale di leggere dati di altri utenti o modificare risorse.»

05:00-09:00 — Human-in-the-Loop. Apri `06_hitl.py`. «L'interrupt sospende il grafo prima dell'azione. Un approvatore umano può scegliere, e il programma riprende con `Command(resume=...)`. Nel demo l'input è locale, non autenticato, quindi non è ancora un sistema di approvazione enterprise.»

09:00-13:00 — Platform. Disegna Git/CI, ECR, EKS/ECS su EC2, IAM, vLLM/Bedrock. «ECR conserva immagini, non le avvia. IAM identifica e autorizza. In Bedrock controlliamo credenziali, region e accesso al modello separatamente. Il principio del minimo privilegio vale per agenti e applicazioni.»

Risorsa: https://www.deeplearning.ai/courses/safe-and-reliable-ai-via-guardrails

18 Videolezione 07 — Prompt caching e prestazioni

Durata suggerita: 9-12 minuti. **Riprese:** timeline prefill/decode, demo stream e due esecuzioni server.

00:00-03:00 — Il vocabolario delle cache. «KV cache, prefix caching e semantic response cache sono tre strumenti diversi. La KV cache mantiene rappresentazioni nel corso dell'inferenza. Prefix caching può riusare il prefill tra richieste con un prefisso identico. Semantic caching restituisce una risposta già pronta, con rischi di obsolescenza e isolamento dei dati.»

03:00-06:00 — Prefissi identici. Mostra due prompt con system prompt uguale e due domande diverse. «Se cambia la prima parte, il riuso diminuisce. Un prefix cache hit accelera soprattutto l'elaborazione del prompt, non automaticamente la generazione dei nuovi token.»

06:00-09:00 — Demo. Esegui `07_prefix_cache_lab.py`. Mostra i TTFT, senza promettere speedup fissi. Riavvia il server, se possibile, con il parametro di caching disattivato/attivato e ripeti prove. «Non basta osservare un tempo inferiore: bisogna controllare misure ripetute e metriche di cache hit.»

09:00-12:00 — Bedrock. «Il caching dei provider gestiti usa condizioni e prezzi specifici del modello. Un prompt statico lungo può essere memorizzato, ma TTL, token minimi e costo cache-write vanno verificati. Il criterio non è avere cache, è ridurre davvero costi e latenza.»

Risorsa: https://www.deeplearning.ai/courses/building-toward-computer-use-with-anthropic

19 Videolezione 08 — Troubleshooting di un sistema LLM

Durata suggerita: 9-12 minuti. **Riprese:** shell e log in zoom, scheda incident.

00:00-03:00 — Diagnosi per livelli. «Quando una risposta fallisce non bisogna reinstallare tutto. Separiamo GPU, runtime, model loading, endpoint REST, orchestrazione, retrieval e qualità della generazione. Per ciascun livello cerchiamo un'osservazione falsificabile.»

03:00-05:00 — Un errore realmente visto. «Il comando `vllm download-kernels` è stato respinto dalla CLI: ciò non implica server rotto. `vllm --version` e `vllm --help` sono strumenti per riconciliare una guida con la versione installata.»

05:00-08:00 — Errore vs risposta incompleta. «Un `finish_reason=length` indica il limite token, non un errore HTTP. Un `CUDA OOM` richiede analizzare VRAM, contesto e concorrenza. Un errore di credenziali Bedrock deve essere verificato dentro il container o processo dove gira il codice, non soltanto nel terminale dell'host.»

08:00-12:00 — Esercizio. Esegui `08_diagnostica.py` prima a server attivo, poi con porta volutamente errata tramite `VLLM_URL=http://127.0.0.1:9999/v1`. Crea un incident report: sintomo, ipotesi, esperimento, correzione, test regressione.

20 Videolezione 09 — Progetto finale e discussione tecnica

Durata suggerita: 10-12 minuti. **Riprese:** architettura completa, esecuzione end-to-end e tabella metriche.

00:00-03:00 — Demo della soluzione. «Un assistente in ingresso non dovrebbe eseguire direttamente qualsiasi comando. La richiesta va al router, poi al motore di ricerca documentale o al modello generativo. Gli output sono verificati rispetto alle fonti. Se esiste un'azione sensibile, attiviamo HITL.»

03:00-06:00 — Tracciamento. Mostra un record fittizio con ID richiesta, modello, source IDs, token, latenza e stato approvazione. «Questo permette di diagnosticare problemi e valutare regressioni nel tempo. La presenza di dati personali impone protezioni e minimizzazione dei log.»

06:00-09:00 — Valutazione. «Mostriamo HitRate@k e controllo manuale delle risposte, misure P95 e casi senza fonte. Il sistema non è pronto alla produzione solo perché ha risposto bene tre volte. Serve un insieme rappresentativo di casi, versionamento ed escalation per errori.»

09:00-12:00 — Simulazione di colloquio. Proponi cinque domande: modello installato, GPU, LoRA effettivamente eseguito, RAG utilizzato e suo evaluation framework, server di inferenza. Distingui con precisione ciò che hai provato, progettato e gestito in produzione.

21 Calendario delle sei settimane

Settimana	Studio guidato	Laboratorio	Evidenza da consegnare
1	Modulo 1 + Modulo 2 (parte I)	API vLLM/Qwen, lettura usage, GPU	Screenshot/log `vllm --version`, `nvidia-smi`, richiesta REST
2	Modulo 2 (parte II) + Modulo 3 (parte I)	Concorrenza, TTFT, ingestione KB	Tabella latenza e documenti indicizzati
3	Modulo 3 (parte II) + Modulo 4 (parte I)	Retrieval evaluation, decisione RAG vs FT	HitRate@1/3 e due casi errati annotati
4	Modulo 4 (parte II) + Modulo 5 (parte I)	LoRA opzionale, primo grafo	Report training o motivazione del mancato training; grafo nodi
5	Modulo 5 (parte II) + Modulo 6	Tool contracts, routing, HITL, sicurezza	Ciclo agente limitato e caso approvato/rifiutato
6	Modulo 7 + Modulo 8 + Modulo 9	Prefix cache, incidente simulato e capstone	Report comparativo, log diagnostico e presentazione finale

21.1 Rubrica di verifica individuale (0–4)

0: risposta mancante o sbagliata. 1: definizione vaga, senza terminologia precisa. 2: concetti corretti ma implementazione incompleta. 3: implementazione riproducibile con motivazioni. 4: implementazione documentata, valutata e con limiti/rischi esplicitati.

21.2 Domande trasversali per l'esame

1. Perché una quantizzazione 4-bit non riduce automaticamente tutta la VRAM a un quarto?
2. Che cosa distingue vLLM da una libreria di inferenza Transformers?
3. Se il retrieval ha Recall@5 alto ma risposte scorrette, dove cerchi il problema?
4. Quando useresti fine-tuning invece di RAG? E quando entrambi?
5. Spiega la separazione tra decisione dell'LLM e invocazione effettiva di un tool.
6. Come riprendi un agente dopo un'interruzione e cosa non deve essere ripetuto?
7. Che cosa impone least privilege in un agente con accesso a database e email?
8. In quale situazione un document retrieved diventa vettore d'attacco per prompt injection?
9. Come verifichi un reale cache hit invece di limitarti a misurare un tempo più breve?
10. Quali evidenze porteresti al colloquio per sostenere "ho costruito un RAG valutato"?

21.3 Registro esperimenti

Per ogni laboratorio conserva: data, comando eseguito, versione Python e librerie, modello, sistema operativo, GPU/VRAM, dataset, seed se presente, durata, metriche, screenshot/log minimizzati, osservazioni e limiti. Evita dati reali e credenziali.

21.4 Risposte professionali alle cinque domande iniziali (scheletro)

LLM installato: "Ho installato ed eseguito Qwen3-0.6B via vLLM 0.31.0 su Linux x86_64. Ho verificato l'inferenza con API REST; tengo distinti test di laboratorio e produzione."

Hardware: "GPU NVIDIA: [modello da `nvidia-smi`]; VRAM: [GB]. Configurazione: [single/multi GPU]." Non riempire i campi senza misurarli.

LoRA/QLoRA: "Ho eseguito personalmente [sì/no]; se sì, modello, dataset, framework, hyperparametri, durata, test set e metriche." Non dichiarare esecuzione se hai soltanto letto o preparato lo script.

RAG: "Ho costruito [demo/progetto effettivamente usato], con embeddings [x], vector store [y], top-k [z], reranking [se presente], test con [n] domande e metriche [HitRate@k, groundedness, latenza]."

Serving: "Ho utilizzato vLLM 0.31.0 con endpoint compatibili OpenAI in locale e ne ho testato le richieste. [Aggiungere benchmark solo dopo averlo eseguito]."

22 Videotutorial verificati e riferimenti tecnici (consultati ottobre 2026)

I video sono ospitati sulle rispettive piattaforme esterne. Alcuni richiedono la creazione di un account o applicano condizioni di accesso proprie; non sono inclusi come file nel kit.

Modulo	Videocorso	Durata indicativa	URL
1	How Transformer LLMs Work — Jay Alammar / Maarten Grootendorst	1h34	https://www.deeplearning.ai/cou transformer-llms-work
2	Fast & Efficient LLM Inference with vLLM — Cedric Clyburn	1h38	https://www.deeplearning.ai/cou and-efficient-llm-inference-with-vllm
2/7	Efficiently Serving LLMs — Travis Addair	2h30	https://www.deeplearning.ai/cou serving-llms
3	Building and Evaluating Advanced RAG — Jerry Liu / Anupam Datta	1h55	https://www.deeplearning.ai/cou evaluating-advanced-rag
4	Finetuning Large Language Models — Sharon Zhou	1h25	https://www.deeplearning.ai/cou large-language-models
5/6	Introduction to LangGraph — LangChain Academy	~6h	https://academy.langchain.com/ to-langgraph
5/6	AI Agents in LangGraph — Harrison Chase / Rotem Weiss	1h32	https://www.deeplearning.ai/cou agents-in-langgraph
6	Safe and Reliable AI via Guardrails — Shreya Rajpal	1h32	https://www.deeplearning.ai/cou and-reliable-ai-via-guardrails
7	Building toward Computer Use with Anthropic — lezione Prompt Caching	12m per lezione	https://www.deeplearning.ai/cou toward-computer-use-with-anthropic

22.0.1 Fonti tecniche

- vLLM 0.31.0, API, caratteristiche e benchmark: https://docs.vllm.ai/en/v0.31.0/
- vLLM automatic prefix caching: https://docs.vllm.ai/en/v0.31.0/features/automatic_prefix_caching/
- Hugging Face TRL + PEFT: https://huggingface.co/docs/trl/peft_integration
- Hugging Face AI Agents: https://huggingface.co/learn/agents-course/en/unit0/introduction
- Agentic RAG: https://huggingface.co/learn/agents-course/en/unit3/agentic-rag/agentic-rag
- LangGraph course: https://academy.langchain.com/courses/intro-to-langgraph
- Amazon Bedrock Prompt Caching: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html
- OWASP LLM Prompt Injection: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

22.0.2 Nota di aggiornamento

I comandi in tutorial pubblicati in momenti diversi possono riferirsi a versioni differenti. Per il tuo ambiente usare come riferimento operativo `vllm --version` e `vllm serve --help`. Esempi adatti a Qwen3-0.6B non sono automaticamente validi per ogni altro modello.